

UNIVERSIDAD DE LAS FUERZAS ARMADAS-ESPE

SEDE SANTO DOMINGO DE LOS TSÁCHILAS

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN - DCCO-SS

CARRERA DE INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN



PERIODO : 2026 Abril – Agosto 2026

ASIGNATURA : Programación Avanzada

TEMA : Implementación de un flujo distribuido seguro
con autenticación, roles y registro de eventos

ESTUDIANTE : Lara Anderson

NIVEL-PARALELO - NRC: 29546

DOCENTE : Ing. Geovanni Brito

FECHA DE ENTREGA : 06/07/2026

SANTO DOMINGO – ECUADOR

2026

Tabla de Contenido

Tabla de Contenido.....	2
Objetivo.....	3
Stack Tecnológico.....	3
Base de Datos Seleccionada.....	4
Descripción de la Solución.....	10
Arquitectura.....	10
Estructura del proyecto.....	11
Configuración de Google OAuth.....	12
Generación del JWT.....	14
Validación del JWT.....	14
Reglas de autorización por ruta.....	15
Revocación del token.....	15
Roles y permisos.....	17
Diagrama del proceso.....	18
Flujo de compra o alquiler.....	20
Transacciones utilizadas.....	21
Validaciones implementadas.....	22
Manejo centralizado de errores.....	23
Logs en UTC.....	24
Dos principios SOLID.....	25
Single Responsibility Principle (SRP).....	25
Dependency Inversion Principle (DIP).....	26
Patrón de diseño.....	26
Evidencias del cliente web.....	27
Evidencias de la base de datos.....	30
Conclusiones.....	30
Referencias.....	31
Github.....	31

Objetivo

Desarrollar una aplicación distribuida con cliente web y backend que permita ejecutar de manera segura un proceso de compra o alquiler, integrando autenticación con Google, generación y validación de JWT, roles de usuario, control de errores, logs en UTC, principios SOLID y un patrón de diseño estudiado durante el parcial.

Stack Tecnológico

Las tecnologías usadas se resumen en la Tabla 1.

Tabla 1

Detalle del Stack Tecnológico

Capa	Tecnología
Backend	Spring Boot 4.1.0 Java 21
Autenticación Google	google-api-client 2.2.0
JWT	jjwt 0.11.5
Base de datos	MySQL
Contenedores	Docker
Frontend	Flutter Dart
Almacenamiento seguro cliente	flutter_secure_storage
Decodificación JWT en cliente	jwt_decoder
Gestión de estado	Provider ChangeNotifier

Base de Datos Seleccionada

Se usó la base de datos Sakila. Se implementó la Alternativa B, la cual se centra en alquiler y devolución de películas, usando las entidades Film, Inventory, InventoryAvailability, Customer, Rental y Payment, mapeadas mediante JPA.

Se modificó la estructura de la tabla film en la base de datos Sakila mediante la adición de la columna active.

Código 1

Adición de Control de Disponibilidad en la Tabla Film

```
SQL
database/sakila-modity.sql

ALTER TABLE film
  ADD COLUMN active BOOLEAN NOT NULL DEFAULT TRUE AFTER rating;
```

Como se muestra en el Código 2, se creó la tabla app_user. La estructura fue definida con la asignación de una clave primaria, restricciones de unicidad y el establecimiento de llaves foráneas que la relacionan con las tablas preexistentes customer y staff.

Código 2

Creación de la Tabla de Usuarios para Autenticación OAuth

```
SQL
database/sakila-modity.sql

CREATE TABLE app_user (
  user_id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  google_id        VARCHAR(255) NOT NULL,           -- "sub" devuelto por Google
  email            VARCHAR(150) NOT NULL,
  full_name        VARCHAR(150) NOT NULL,
  role             ENUM('ADMIN', 'CUSTOMER') NOT NULL DEFAULT 'CUSTOMER',
  customer_id      SMALLINT UNSIGNED DEFAULT NULL, -- si role = CUSTOMER
  staff_id         TINYINT UNSIGNED DEFAULT NULL,  -- si role = ADMIN
  active           BOOLEAN NOT NULL DEFAULT TRUE,
  token_version    INT UNSIGNED NOT NULL DEFAULT 1, -- invalida todos los JWT previos al incrementarse
  created_at       DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```

last_login    DATETIME DEFAULT NULL,
PRIMARY KEY (user_id),
UNIQUE KEY uq_google_id (google_id),
UNIQUE KEY uq_email (email),
KEY idx_fk_customer_id (customer_id),
KEY idx_fk_staff_id (staff_id),
CONSTRAINT fk_appuser_customer FOREIGN KEY (customer_id)
REFERENCES customer (customer_id) ON DELETE SET NULL ON UPDATE CASCADE,
CONSTRAINT fk_appuser_staff FOREIGN KEY (staff_id)
REFERENCES staff (staff_id) ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Como se muestra en el Código 3, se creó la tabla `revoked_token`. La estructura fue definida con la asignación de una clave primaria, la creación de un índice y el establecimiento de una llave foránea con restricciones de actualización y eliminación en cascada que la relaciona con la tabla `app_user`.

Código 3

Creación de la Tabla de Tokens Revocados

```

SQL
database/sakila-modify.sql

CREATE TABLE revoked_token (
  jti          VARCHAR(36) NOT NULL,
  user_id      BIGINT UNSIGNED NOT NULL,
  revoked_at   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  expires_at   DATETIME NOT NULL,  -- permite purgar filas ya vencidas
  PRIMARY KEY (jti),
  KEY idx_fk_user_id (user_id),
  CONSTRAINT fk_revoked_user FOREIGN KEY (user_id)
REFERENCES app_user (user_id) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Como se muestra en el Código 4, se creó la tabla `active_session`. La estructura fue definida con la asignación de una clave primaria, la creación de un índice y el establecimiento de una llave foránea con restricciones de actualización y eliminación en cascada que la relaciona con la tabla preexistente `app_user`.

Código 4

Creación de la Tabla de Sesiones Activas

SQL

database/sakila-modify.sql

```
CREATE TABLE active_session (  
  session_id VARCHAR(36) NOT NULL, -- jti del access token  
  user_id BIGINT UNSIGNED NOT NULL,  
  issued_at DATETIME NOT NULL,  
  expires_at DATETIME NOT NULL,  
  ip_address VARCHAR(45) DEFAULT NULL,  
  user_agent VARCHAR(255) DEFAULT NULL,  
  PRIMARY KEY (session_id),  
  KEY idx_fk_user_id (user_id),  
  CONSTRAINT fk_session_user FOREIGN KEY (user_id)  
    REFERENCES app_user (user_id) ON DELETE CASCADE ON UPDATE CASCADE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Se creó la tabla `audit_log`. La estructura fue definida con la asignación de una clave primaria autoincrementable, la creación de múltiples índices y el establecimiento de una llave foránea con restricciones de actualización en cascada y de asignación de valor nulo por eliminación, que la relaciona con la tabla preexistente `app_user`. Como se muestra en el Código 5.

Código 5

Creación de la Tabla de Registro de Auditoría

SQL

database/sakila-modify.sql

```
CREATE TABLE audit_log (  
  log_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,  
  event_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP, -- en UTC  
  level ENUM('INFO', 'WARN', 'ERROR') NOT NULL DEFAULT 'INFO',  
  event_type VARCHAR(50) NOT NULL,  
  user_id BIGINT UNSIGNED DEFAULT NULL,  
  module VARCHAR(100) DEFAULT NULL, -- ruta o módulo, ej. /api/rentals  
  result VARCHAR(20) DEFAULT NULL, -- SUCCESS / FAILURE  
  message VARCHAR(255) DEFAULT NULL, -- mensaje breve, nunca datos sensibles  
  PRIMARY KEY (log_id),  
  KEY idx_fk_user_id (user_id),  
  KEY idx_event_type (event_type),  
  CONSTRAINT fk_audit_user FOREIGN KEY (user_id)  
    REFERENCES app_user (user_id) ON DELETE SET NULL ON UPDATE CASCADE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Se creó una vista en el esquema de la base de datos estructurada para seleccionar múltiples atributos de una entidad principal. Su definición incorpora la evaluación de una condición lógica combinada con una subconsulta de exclusión para generar un nuevo campo calculado dinámicamente, tal como se detalla en el Código 6.

Código 6

Creación de la Vista de Disponibilidad de Inventario

```
SQL
database/sakila-modity.sql

CREATE OR REPLACE VIEW inventory_availability AS
SELECT
    i.inventory_id,
    i.film_id,
    i.store_id,
    (i.active = TRUE AND NOT EXISTS (
        SELECT 1 FROM rental r
        WHERE r.inventory_id = i.inventory_id
            AND r.return_date IS NULL
    )) AS available
FROM inventory i;
```

Como se muestra en el Código 7, se creó un índice compuesto sobre una tabla preexistente, estructurando dicha indexación a partir de múltiples atributos.

Código 7

Creación de Índice Compuesto

```
SQL
database/sakila-modity.sql

CREATE INDEX idx_rental_inventory_open
```

```
ON rental (inventory_id, return_date);
```

Se alteró la estructura de una tabla existente mediante la modificación de las restricciones de obligatoriedad y los parámetros de datos en columnas previas. Adicionalmente, se agregaron múltiples columnas nuevas, las cuales fueron definidas con valores por defecto, controles de nulidad y posiciones específicas dentro del esquema de la entidad. Como se muestra en el Código 8.

Código 8

Alteración Estructural de Tabla de Usuarios

SQL

database/sakila-modify.sql

```
ALTER TABLE app_user MODIFY COLUMN role ENUM('ADMIN', 'CUSTOMER') NOT NULL DEFAULT 'CUSTOMER';
ALTER TABLE app_user
  MODIFY COLUMN google_id VARCHAR(255) NULL;
ALTER TABLE app_user
  ADD COLUMN password_hash VARCHAR(255) NULL AFTER full_name;
ALTER TABLE app_user
  ADD COLUMN auth_provider ENUM('GOOGLE', 'LOCAL') NOT NULL DEFAULT 'LOCAL' AFTER password_hash;
ALTER TABLE app_user
  ADD COLUMN status ENUM('PENDING', 'ACTIVE') NOT NULL DEFAULT 'PENDING' AFTER auth_provider,
  ADD COLUMN verification_code VARCHAR(255) NULL AFTER status,
  ADD COLUMN verification_code_expires_at DATETIME NULL AFTER verification_code;
```

Se realizaron múltiples alteraciones estructurales en diversas tablas preexistentes de la base de datos. Estas modificaciones consistieron en la adición de nuevas columnas con restricciones de nulidad y valores por defecto, la eliminación de atributos previamente definidos y la modificación de las reglas de obligatoriedad en campos existentes. Como se muestra en el Código 9.

Código 9

Alteraciones Estructurales Múltiples en Tablas Existentes

SQL

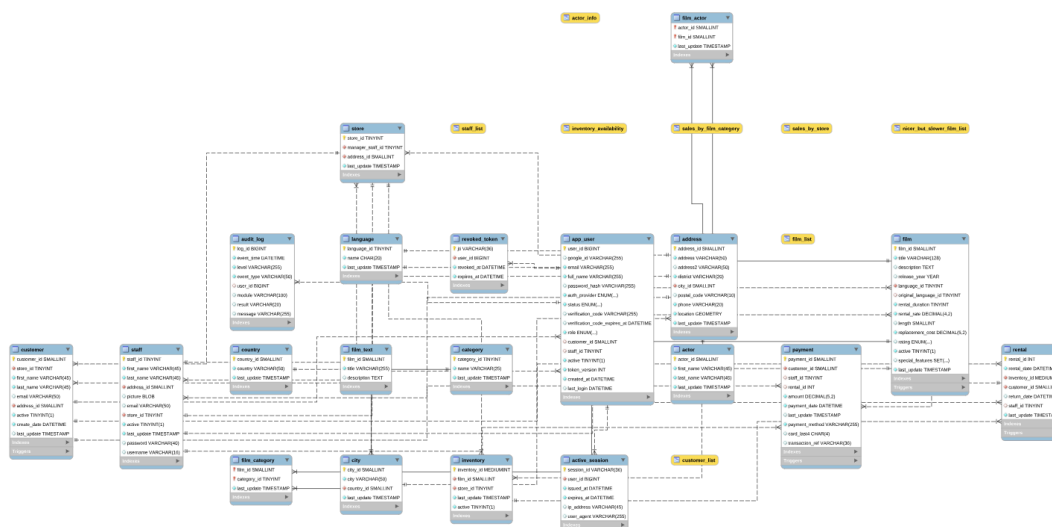
database/sakila-modify.sql

```
ALTER TABLE inventory ADD COLUMN active BOOLEAN NOT NULL DEFAULT TRUE;
ALTER TABLE staff DROP COLUMN username;
ALTER TABLE staff DROP COLUMN password;
ALTER TABLE rental MODIFY staff_id TINYINT UNSIGNED NULL;
ALTER TABLE payment MODIFY staff_id TINYINT UNSIGNED NULL;
ALTER TABLE payment
  ADD COLUMN payment_method VARCHAR(20) NOT NULL DEFAULT 'CARD',
  ADD COLUMN card_last4 CHAR(4) DEFAULT NULL,
  ADD COLUMN transaction_ref VARCHAR(36) DEFAULT NULL;
```

Como se muestra en la Figura 1, se generó el diagrama entidad relación que representa el esquema estructural final de la base de datos. La representación visualiza el conjunto de entidades, las vistas incorporadas y las relaciones de cardinalidad establecidas entre ellas mediante llaves primarias y foráneas.

Figura 1

Diagrama Entidad-Relación de la Base de Datos Final



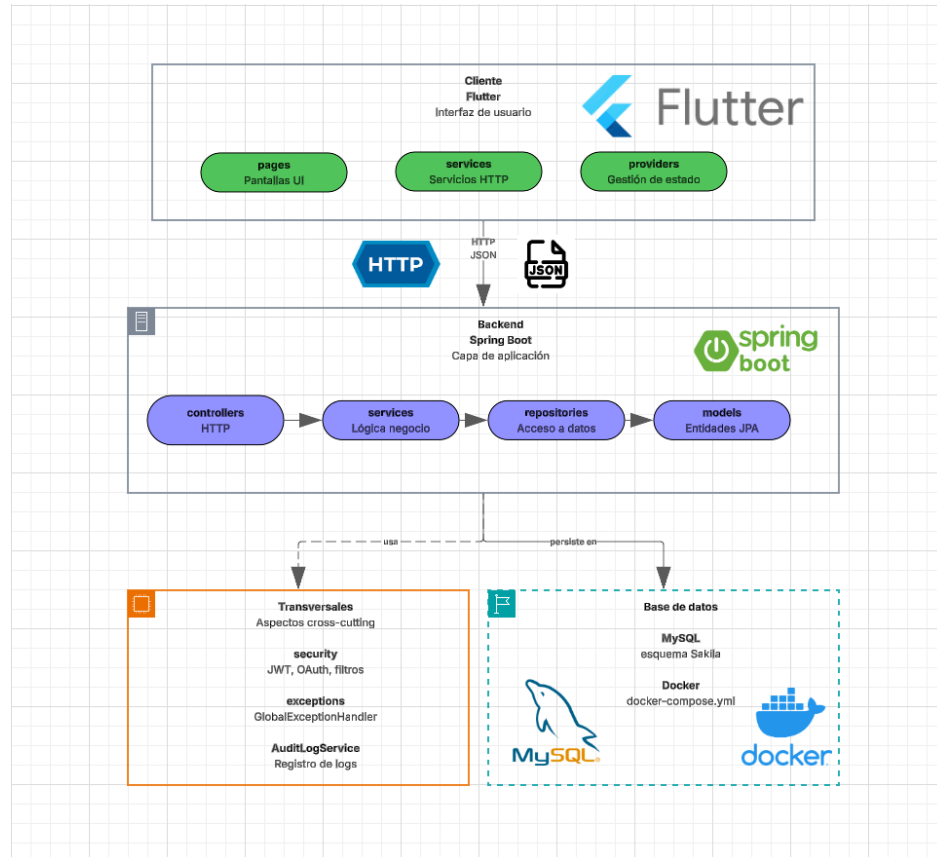
Descripción de la Solución

La aplicación permite que un cliente autenticado explore el catálogo de películas, filtre por categoría, clasificación y año, alquile un ejemplar disponible pagando con una tarjeta simulada, consulte su historial y registre la devolución. El administrador gestiona películas, inventario, revisar todos los alquileres y auditar los eventos del sistema. La seguridad de acceso se resuelve enteramente en el backend mediante un filtro JWT y reglas de autorización por ruta, de modo que ocultar opciones en el cliente no es la única barrera de protección.

Arquitectura

Figura 2

Arquitectura del sistema



Fuente: [Elaboración Propia](#)

Estructura del proyecto

Código 10

Estructura del Proyecto

SQL

```

project_LaraAnderson
├─ README.md
├─ backend
│  ├─ pom.xml
│  └─ src
│     └─ main
│        ├── java/com/example/demo
│        │  ├── DemoApplication.java
│        │  ├── authentication/
│        │  ├── configuration/
│        │  ├── controllers/
│        │  ├── database/
│        │  └── errors/
  
```



```

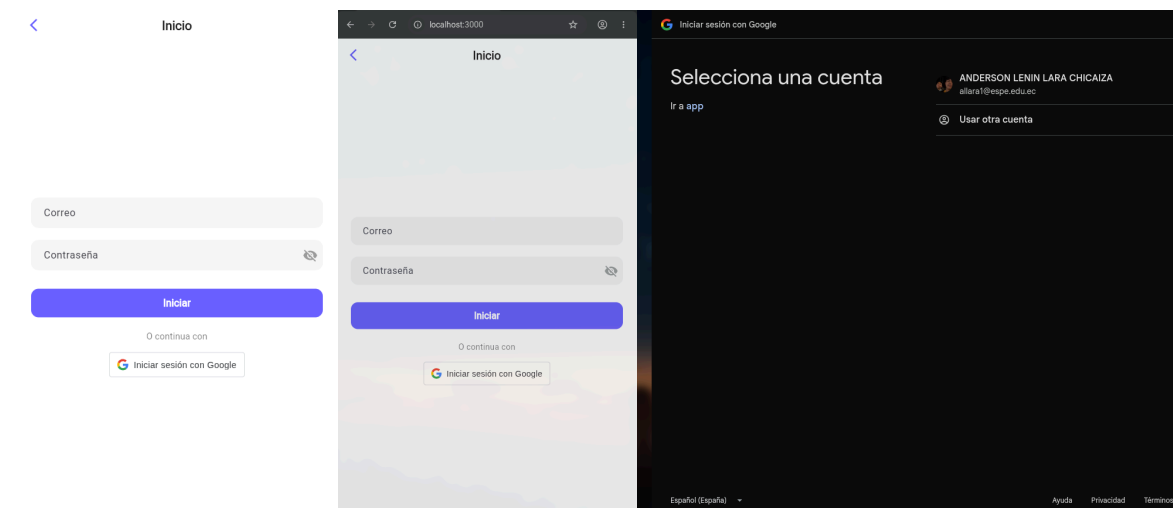
GoogleIdToken idToken = verifier.verify(idTokenString);
if (idToken != null) {
    return idToken.getPayload();
} else {
    throw new IllegalArgumentException("Invalid ID token.");
}
}

```

La verificación del idToken emitido por Google se realiza estrictamente en el backend, nunca debemos confiar en los datos que el cliente envía directamente sin validar. Para esto, la clase GoogleTokenVerifier configura una instancia de GoogleIdTokenVerifier utilizando la audiencia (Client ID) cargada desde las variables de entorno. Su función principal es validar la firma y el emisor del token antes de proceder a extraer el payload. El flujo completo es el siguiente: el cliente obtiene un idToken de Google, lo envía mediante un POST a la ruta /api/auth/google, el método AuthService.authenticateWithGoogle() verifica el token, busca o crea el AppUser correspondiente, determina su rol y genera el JWT propio de la aplicación, el cual finalmente se devuelve al cliente dentro de un AuthResponse.

Figura 3

Inicio de sesión por OAuth



Generación del JWT

La clase `JwtTokenProvider` construye el token utilizando la librería `jjwt`, el cual es firmado con HMAC-SHA256 mediante una clave secreta que se carga desde una variable de entorno denominada `jwt.secret`. Este token incluye además un identificador único `jti` generado con `UUID`, permitiendo así aplicar la estrategia de revocación descrita en la sección 10.

Código 12

Generación del token

```
Java
public String generateToken(AppUser user) {
    Date now = new Date();
    Date expiryDate = new Date(now.getTime() + expiration);

    Map<String, Object> claims = new HashMap<>();
    claims.put("id", user.getId());
    claims.put("email", user.getEmail());
    claims.put("role", user.getRole().name());
    claims.put("fullName", user.getFullName());

    return Jwts.builder()
        .setClaims(claims)
        .setSubject(Long.toString(user.getId()))
        .setIssuedAt(now)
        .setExpiration(expiryDate)
        .setId(UUID.randomUUID().toString()) // jti - clave para revocación
        .signWith(key, SignatureAlgorithm.HS256)
        .compact();
}
```

Validación del JWT

`JwtAuthenticationFilter` extiende `OncePerRequestFilter` y se ejecuta antes que `UsernamePasswordAuthenticationFilter`. Extrae el token del header `Authorization`, valida su firma y expiración con `tokenProvider.getClaims()`, y adicionalmente comprueba si el `jti` aparece en la tabla `revoked_token`.

Código 13

Validación y captura de errores

```
Java
Claims claims = tokenProvider.getClaims(token);
String jti = claims.getId();
if (jti != null && revokedTokenRepository.existsByJti(jti)) {
    logger.warn("AUDIT_LOG: Token revocado (jti: {})", jti);
    request.setAttribute("jwt_error", "Token revocado");
} else {
    String roleStr = claims.get("role", String.class);
    ...
    SecurityContextHolder.getContext().setAuthentication(authentication);
}
// Excepciones controladas explícitamente:
// ExpiredJwtException "Token vencido"
// SignatureException MalformedJwtException token inválido/alterado
```

Las peticiones sin token válido llegan sin autenticación al SecurityContextHolder; SecurityConfig las rechaza en la fase de autorización mediante JwtAuthenticationEntryPoint (401) o CustomAccessDeniedHandler (403), según corresponda.

Reglas de autorización por ruta

Código 14

Reglas de autorización

```
Java
.requestMatchers("/api/auth/logout").authenticated()
.requestMatchers("/api/auth/**").permitAll()
.requestMatchers(HttpMethod.GET, "/api/films/**").hasAnyRole("CUSTOMER", "ADMIN")
.requestMatchers("/api/films/**").hasRole("ADMIN") // crear/editar solo admin
.requestMatchers("/api/inventory/**").hasRole("ADMIN")
.requestMatchers("/api/rentals/**").hasAnyRole("CUSTOMER", "ADMIN")
.requestMatchers("/api/admin/**").hasRole("ADMIN")
.requestMatchers("/api/user/**").hasAnyRole("CUSTOMER", "ADMIN")
.anyRequest().authenticated()
```

Revocación del token

La estrategia elegida es una lista de tokens revocados identificados por jti (JWT ID), persistida en la tabla revoked_token. Al cerrar sesión, el backend inserta el jti del token actual en

esta tabla junto con su fecha de expiración; en cada petición posterior, JwtAuthenticationFilter comprueba su existencia antes de autorizar la petición

Código 15

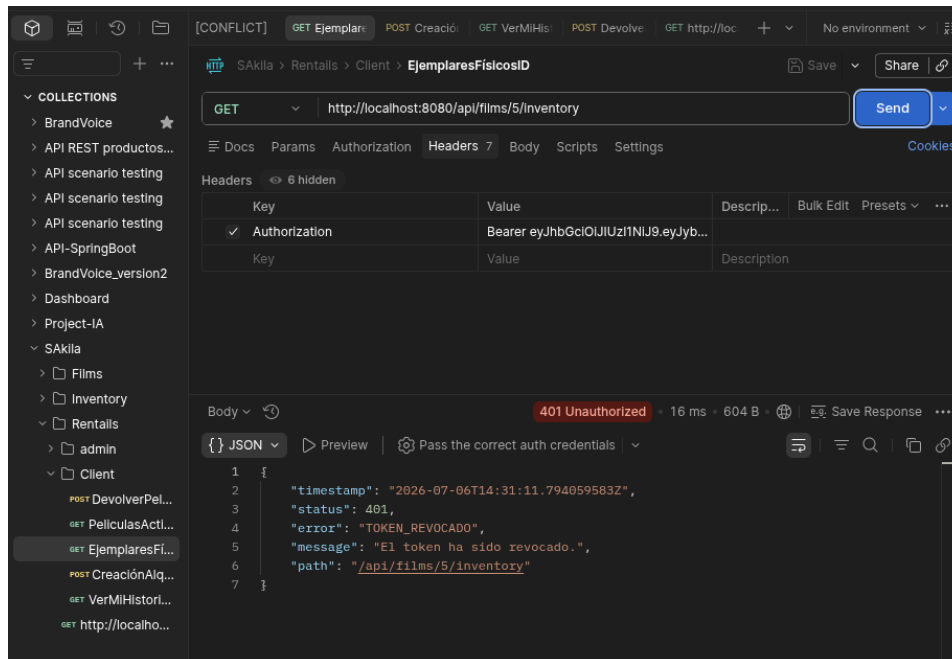
Modelo de persistencia de revocación

```
Java
@Entity
@Table(name = "revoked_token")
public class RevokedToken {
    @Id
    @Column(name = "jti", length = 36, nullable = false)
    private String jti;
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private AppUser user;
    private LocalDateTime revokedAt = LocalDateTime.now();
    private LocalDateTime expiresAt;
}
```

El repositorio expone existsByJti(jti) para la verificación O(1) en cada request, y deleteByExpiresAtBefore(now) para la limpieza de tokens revocados ya vencidos (evita crecimiento indefinido de la tabla). El campo tokenVersion en AppUser existe como mecanismo adicional disponible, aunque la estrategia principal activa es la lista de revocación por jti.

Figura 4

Intento de reutilización de un token revocado (401)



Roles y permisos

El enum Role vive en el modelo AppUser (ADMIN, CUSTOMER) y se asigna en el servidor al crear o consultar el usuario tras la autenticación — nunca se acepta el rol enviado desde el cliente.

Tabla 2

Tabla Resumen de Roles y Permisos

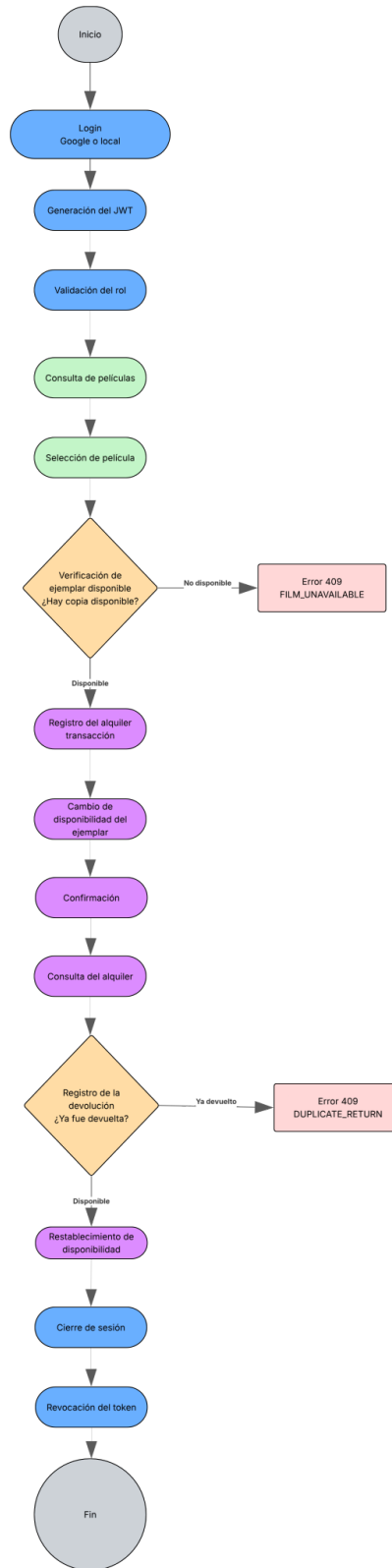
Petición	Descripción
GET /api/films/	- Acceso de ADMIN y CUSTOMER. - Permite consultar, listar o ver los detalles de las películas disponibles en el catálogo.
POST /api/films/	- Acceso únicamente de ADMIN. - Permite registrar o añadir una nueva película al catálogo del sistema.
PUT /api/films/	- Acceso únicamente de ADMIN.

DELETE /api/films/	- Permite actualizar o editar la información de una película ya existente. - Acceso únicamente de ADMIN.
ANY /api/inventory/	- Permite eliminar una película de la base de datos de forma permanente. - Acceso únicamente de ADMIN. - Permite gestionar el control de inventario, stock y copias físicas/digitales de las películas.
ANY /api/rentals/	Acceso para ADMIN y CUSTOMER. Permite gestionar el flujo de alquileres de películas: crear rentas, consultar el historial de alquileres o registrar devoluciones.
ANY /api/admin/	- Acceso únicamente de ADMIN. - Permite el acceso a endpoints restringidos de administración global y configuraciones críticas del sistema.
POST /api/auth/logout	- Cualquier usuario Autenticado. - Permite cerrar la sesión del usuario actual e invalidar su token en el backend, revocación por jti.

Diagrama del proceso

Figura 5

Diagrama del proceso

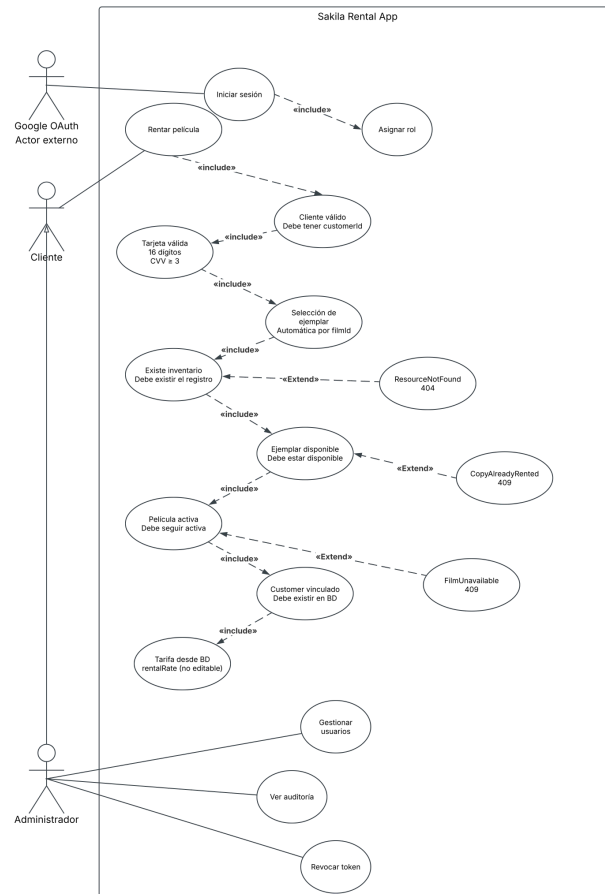


Fuente: [Elaboración Propia](#)

Flujo de compra o alquiler

Figura 6

Flujo de compra o Alquiler



Fuente: [Elaboración Propia](#)

Código 16

Validación de disponibilidad y tarifa

```
Java
if (!availability.getAvailable()) {
```

```

        throw new CopyAlreadyRentedException("El ejemplar ya se encuentra alquilado o no disponible");
    }
}
if (!inventory.getFilm().getIsActive()) { // Nota: en la imagen se lee getActive() pero con un espacio, se
transcribe literal: getActive()
if (!inventory.getFilm().getIsActive()) {
    throw new FilmUnavailableException("La película no está activa en el catálogo");
}
BigDecimal amount = inventory.getFilm().getRentalRate();
if (amount == null) {
    throw new InvalidAmountException("CANTIDAD_INVALIDA", "La película no tiene tarifa configurada");
}
}
}

```

Transacciones utilizadas

Todos los métodos que modifican estado en RentalService, InventoryService y AuditLogService están anotados con `@Transactional` de Spring, garantizando atomicidad: si ocurre una excepción durante el registro del alquiler, el cambio de disponibilidad del ejemplar y el registro del pago simulado, la transacción completa se revierte y no queda un estado inconsistente en la base de datos.

Código 17

Anotación transaccional

```

Java
@Transactional
public RentalDTO createRental(RentalRequestDTO request, Long appId) {
    AppUser appUser = appUserRepository.findById(appId)
        .orElseThrow(() -> new ResourceNotFoundException("Usuario no encontrado"));
    ...
    Inventory inventory = inventoryRepository.findById(inventoryId)
        .orElseThrow(() -> new ResourceNotFoundException("El inventario no existe"));
    Customer customer = customerRepository.findById(appUser.getId())
        .orElseThrow(() -> new ResourceNotFoundException("Perfil de cliente no encontrado"));
    BigDecimal amount = inventory.getFilm().getRentalRate();
    ...
    String transactionRef;
    try {
        transactionRef = paymentGatewayService.processPayment(
            request.getCardNumber(), request.getCardHolder(),

```

```

        request.getExpirationDate(), request.getCvv(), amount
    );
    auditLogService.logEvent("INFO", "PURCHASE_CREATED", appId,
        "/api/rentals", "SUCCESS", "Amount: " + amount + ", Ref: " + transactionRef);
} catch (PaymentDeclinedException e) {
    auditLogService.logEvent("WARN", "PAYMENT_DECLINED", appId,
        "/api/rentals", "FAILURE", "Reason: " + e.getMessage());
    throw e; // @Transactional revierte: no queda Rental ni Payment ni cambio de disponibilidad
}
..
Rental rental = new Rental();
rental.setRentalDate(LocalDate.now());
rental.setInventory(inventory);
rental.setCustomer(customer);
rental = rentalRepository.save(rental);
Payment payment = new Payment();
payment.setCustomer(customer);
payment.setRental(rental);
payment.setAmount(amount);
payment.setPaymentMethod("CARD");
payment.setCardLast4(request.getCardNumber().substring(request.getCardNumber().length() - 4));
payment.setTransactionRef(transactionRef);
paymentRepository.save(payment);
auditLogService.logEvent("INFO", "RENTAL_CREATED", appId, "/api/rentals",
    "SUCCESS", "Rental ID: " + rental.getId());

rental.setPayments(List.of(payment));
return new RentalDTO(rental);
}

```

Validaciones implementadas

Tabla 3

Tabla de resumen de validaciones

Excepción	Código	Caso
ResourceNotFoundException	404	Película, cliente, inventario o alquiler inexistente
CopyAlreadyRentedException	409	Ejemplar ya alquilado
FilmUnavailableException	409	Película inactiva en el

		catálogo
DuplicateReturnException	409	Devolución ya registrada previamente
InsufficientStockException	409	Sin ejemplares disponibles para la película
InvalidAmountException	400	Tarjeta, CVV y tarifa inválidos
PaymentDeclinedException	402 409	Pago rechazado
MethodArgumentNotValidException	400	Campos incompletos o formato inválido

Manejo centralizado de errores

GlobalExceptionHandler, anotado con `@RestControllerAdvice`, centraliza el tratamiento de excepciones para todos los controladores. Construye una estructura uniforme y registra el evento en el log antes de responder.

Código 18

Estructura uniforme de error

```

Java
private Map<String, Object> createErrorBody(HttpStatus status, String error, String message, String path)
{
    Map<String, Object> body = new LinkedHashMap<>();
    body.put("timestamp",
        ZonedDateTime.now(ZoneId.of("UTC")).format(DateTimeFormatter.ISO_INSTANT));
    body.put("status", status.value());
    body.put("error", error);
    body.put("message", message);
    body.put("path", path);
    return body;
}

```

Ejemplo real de respuesta generada por el handler ante un ejemplar ya alquilado:

Java

```
{
  "timestamp": "2026-07-05T23:10:00Z",
  "status": 409,
  "error": "COPY_ALREADY_RENTED",
  "message": "El ejemplar ya se encuentra alquilado o no disponible",
  "path": "/api/rentals"
}
```

Los errores de base de datos (`DataAccessException`) se capturan y se responde un mensaje genérico al cliente, mientras el detalle técnico solo se imprime en el log del servidor — nunca se exponen consultas SQL, trazas ni cadenas de conexión al usuario final.

Logs en UTC

`AuditLogService` centraliza el registro de eventos de negocio (alquiler, devolución, cambios de stock) persistiéndolos en la tabla `audit_log` y además imprimiéndolos en consola con timestamp en UTC. `GlobalExceptionHandler` aplica el mismo criterio para errores.

Código 19

Registro de eventos en UTC

Java

```
String timestamp = ZonedDateTime.now(ZoneId.of("UTC")).format(DateTimeFormatter.ISO_INSTANT);
// Formato: 2026-07-01T15:30:00Z INFO PURCHASE_CREATED user=25 order=103
System.out.printf("%s %s %s user=%s %s\n", timestamp, level, eventType, userStr, message);
```

El registro nunca incluye contraseñas, el JWT completo, secretos ni cadenas de conexión únicamente identificadores (`userId`, entidad afectada) y un mensaje breve del resultado.

Figura 7

Logs del backend en consola, formato UTC


```

backend > logs > backend-sakila.log
115 2026-07-06 10:30:18.804 [main] INFO o.hibernate.orm.connections.pooling - HHH10001005: Database info:
120 Default catalog/schema: sakila/undefined
121 Autocommit mode: undefined/unknown
122 Isolation level: REPEATABLE_READ (default REPEATABLE_READ)
123 JDBC fetch size: none
124 Pool: DataSourceConnectionProvider
125 Minimum pool size: undefined/unknown
126 Maximum pool size: undefined/unknown
127 2026-07-06 10:30:20.267 [main] INFO org.hibernate.orm.core - HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)
128 2026-07-06 10:30:20.536 [main] INFO o.s.o.j.localContainerEntityManagerFactoryBean - Initialized JPA EntityManagerFactory for persistence unit 'default'
129 2026-07-06 10:30:20.753 [main] INFO o.s.d.j.r.q.QueryEnhancerFactories - Hibernate is in classpath; If applicable, HQL parser will be used.
130 2026-07-06 10:30:22.672 [main] WARN o.s.b.j.a.jpaBaseConfiguration$JpaWebConfiguration - spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. Explicitly configure it instead.
131 2026-07-06 10:30:22.850 [main] WARN o.s.b.s.a.UserDetailsServiceAutoConfiguration -
132
133 Using generated security password: 18709d10-6a70-4fd9-8a41-22b75a420358
134
135 This generated password is for development use only. Your security configuration must be updated before running your application in production.
136
137 2026-07-06 10:30:22.870 [main] INFO o.s.s.c.a.c.InitializeUserDetailsServiceBeanManagerConfigurer$InitializeUserDetailsServiceManagerConfigurer - Global AuthenticationManager configured with UserDetailsService bean with name inMemory
138 2026-07-06 10:30:23.550 [main] INFO o.s.boot.tomcat.TomcatWebServer - Tomcat started on port 8080 (http) with context path '/'
139 2026-07-06 10:30:23.570 [main] INFO com.example.demo.DemoApplication - Started DemoApplication in 7.638 seconds (process running for 7.966)
140 2026-07-06 10:30:55.588 [http-nio-8080-exec-1] INFO o.a.c.c.C.[Tomcat].[localhost].[/] - Initializing Spring DispatcherServlet 'dispatcherServlet'
141 2026-07-06 10:30:55.589 [http-nio-8080-exec-1] INFO o.s.web.servlet.DispatcherServlet - Initializing Servlet 'dispatcherServlet'
142 2026-07-06 10:30:55.592 [http-nio-8080-exec-1] INFO o.s.web.servlet.DispatcherServlet - Completed initialization in 3 ms
143 2026-07-06 10:30:56.962 [http-nio-8080-exec-2] WARN c.e.d.s.JwtAuthenticationFilter - AUDIT_LOG: Token revocado (jti: ced62ac8-2347-4b3d-9dce-8aac0af67412)
144

```

Dos principios SOLID

Single Responsibility Principle (SRP)

Componente: separación estricta entre AuthController (HTTP), AuthService (lógica de autenticación/registro) y AppUserRepository (acceso a datos). Ninguna clase mezcla responsabilidades de transporte HTTP, reglas de negocio y persistencia.

Problema que resuelve: sin esta separación, cambios en el protocolo HTTP (por ejemplo, pasar de REST a GraphQL) obligarían a tocar la lógica de negocio. Con SRP, AuthService puede reutilizarse desde cualquier controlador o incluso un job programado sin cambios.

Código 20

AuthController delega en AuthService

```

Java
@PostMapping("/google")
public ResponseEntity<?> authenticateWithGoogle(@RequestBody TokenRequest tokenRequest) {
    AuthResponse response = authService.authenticateWithGoogle(tokenRequest.getIdToken());
    // el controller solo traduce la respuesta a HTTP; toda la lógica vive en AuthService
}

```

Resultado obtenido: los controladores del proyecto (9 en total) son delgados — delegan siempre en la capa de servicios, lo que facilita las pruebas unitarias y el mantenimiento.

Dependency Inversion Principle (DIP)

Componente: todos los servicios dependen de interfaces de repositorio (RentalRepository, AppUserRepository, RevokedTokenRepository, etc.) que extienden JpaRepository, no de implementaciones concretas de acceso a datos. La inyección se realiza por constructor mediante @RequiredArgsConstructor (Lombok). Problema que resuelve: RentalService no conoce ni depende de cómo Spring Data JPA traduce sus métodos a SQL; si mañana se cambiara el motor de persistencia, la interfaz del repositorio permanecería igual.

Código 21

Inyección de dependencias por constructor

```
Java
@Service
@RequiredArgsConstructor
public class RentalService {
    private final RentalRepository rentalRepository;           // interfaz, no implementación
    private final InventoryAvailabilityRepository inventoryAvailabilityRepository;
    private final AuditLogService auditLogService;
    // Spring inyecta las implementaciones concretas en tiempo de ejecución
}
```

Resultado obtenido: alto desacoplamiento entre la lógica de negocio y la tecnología de persistencia; permite además sustituir cualquier repositorio por un mock en pruebas unitarias sin tocar RentalService.

Patrón de diseño

El patrón Proxy está representado por la cadena JwtAuthenticationFilter + SecurityConfig, que actúa como un proxy de seguridad interpuesto entre el cliente HTTP y los controladores reales. Ninguna petición llega directamente al servicio protegido: primero atraviesa el filtro, que decide si autentica, rechaza o deja pasar la solicitud según la validez,

expiración y estado de revocación del JWT, y luego atraviesa la capa de autorización por rol antes de alcanzar el recurso solicitado.

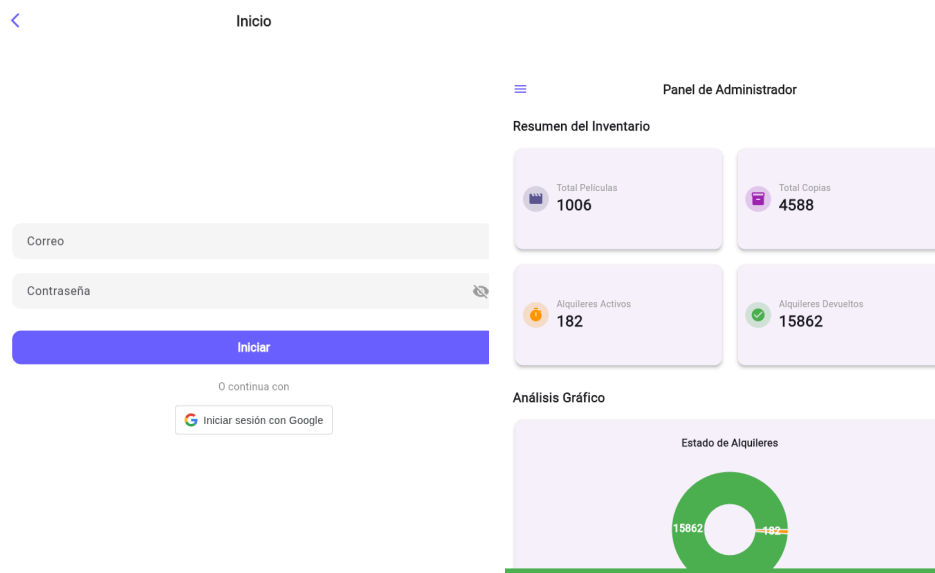
Código 22

Proxy de autenticación previo al servicio real

```
Java
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {
    // Intercepta TODAS las peticiones antes de que lleguen al controlador real
    protected void doFilterInternal(HttpServletRequest request, ...) {
        // 1. Verifica formato y firma del token
        // 2. Verifica si el jti está en la lista de revocados (proxy de control)
        // 3. Solo si pasa, delega el acceso al recurso protegido (filterChain.doFilter)
    }
}
```

Esto coincide exactamente con la definición de Proxy dada en la guía de la tarea: 'Control previo de autenticación y autorización antes de acceder a los servicios'. El controlador (por ejemplo RentalController) nunca necesita saber cómo se validó el token — confía en que, si la petición llega hasta él, ya fue autorizada por el proxy de seguridad.

Evidencias del cliente web



Administrar Peliculas

Buscar pelicula...

Buscar

AAAA
ID: 1006 | Tarifa: \$4.99

Anderson
ID: 1005 | Tarifa: \$4.99

xxxx
ID: 1004 | Tarifa: \$4.99

MBappe egoista
ID: 1003 | Tarifa: \$4.12

Ecuador tu papa
ID: 1002 | Tarifa: \$1.05

AVENGERS ENDGAME: VERSION EXTENDIDA
ID: 1001 | Tarifa: \$4.99

ZORRO ARK
ID: 1000 | Tarifa: \$4.99

ZOOLANDER FICTION
ID: 999 | Tarifa: \$2.99

ZHIVAGO CORE
ID: 998 | Tarifa: \$0.99

Total Ejemplares: 1

Ejemplar ID: 4587
Tienda: 1

Disponible

Catálogo de Películas

Buscar por título...

Bucar

AAAA

2026

\$4.99

1 disp.

Anderson

2026

\$4.99

1 disp.

xxxx

2026

\$4.99

Agotado

MBappe egoista

2010

Ecuador tu papa

2026

AVENGERS ENDGAME: ...

2020

Login exitoso

AAAA

AAAA

☆ Clasificación

g

📅 Año de lanzamiento

2026

🕒 Duración (min)

120

💰 Tarifa de Alquiler

\$4.99

Ejemplares disponibles: 1

Alquilar Ahora

Checkout Seguro

Alquilar: Anderson

Total a pagar: \$4.99

Número de Tarjeta

0/16

Nombre del Titular

Expiración (MM/A...

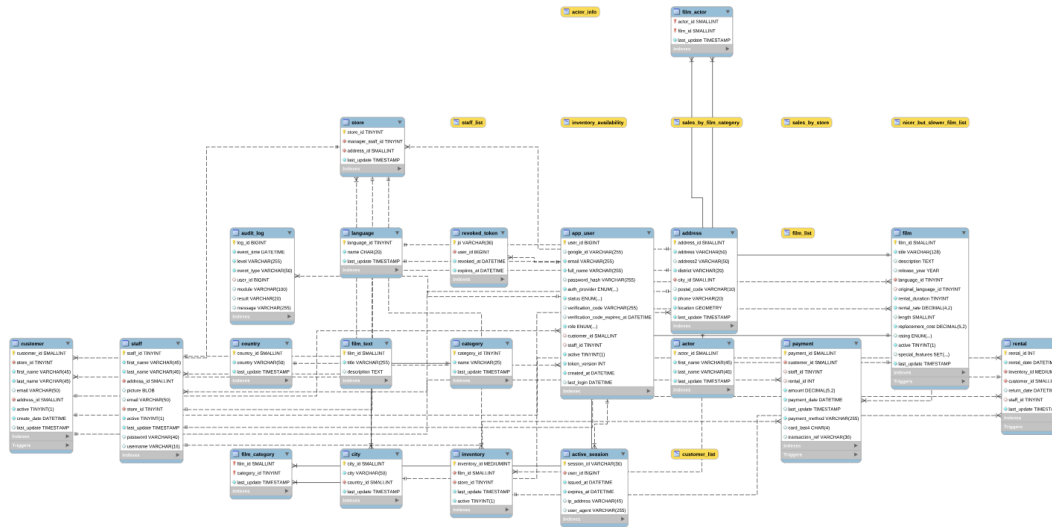
0/5

CVV

0/3

Confirmar Pago y Alquilar

Evidencias de la base de datos



Conclusiones

El backend implementa de forma sólida y verificable en código real los requisitos centrales de la rúbrica: autenticación con Google OAuth, generación y validación de JWT con claims completos, un mecanismo efectivo de revocación basado en jti, autorización por roles a nivel de ruta, manejo centralizado de errores con estructura uniforme, y logs en UTC.

El principio de responsabilidad única y la inversión de dependencias están presentes de forma natural en la separación Controller/Service/Repository del proyecto, y el patrón Proxy queda representado fielmente por la cadena de filtros de seguridad de Spring Security.

El cliente Flutter cubre las 18 pantallas mínimas exigidas, diferenciando la experiencia de administrador y cliente, aunque —como exige la tarea— la seguridad real recae en el backend y no en la interfaz.

Referencias

Spring Security Reference Documentation <https://docs.spring.io/spring-security/reference/>

JWT (Java JWT) <https://github.com/jwt/jwt>

Google Identity Services / Sign-In for Server-Side Apps <https://developers.google.com/identity>

Flutter documentation — <https://docs.flutter.dev/>

Sakila Sample Database — <https://dev.mysql.com/doc/sakila/en/>

Github

<https://github.com/Ander-Lr/project-sakila.git>